# Writing tests that test my code

From SCRUM-2 item 3: moving tests/bronze/test_data_ingestion.py from calling spark.read directly to calling read_raw_file, and writing a parametrized empty-file test across all 9 tables. Follows naming_and_formatting_learnings_2026-11-04.pdf.

## 01 Test your code, not the library

The first three tests called spark.read directly with their own options. They proved Spark can parse CSV -- not that read_raw_file does. If someone removed multiline from read_raw_file, all three would still pass.

After switching the tests to call read_raw_file, the embedded-newline test became the real regression guard. The 'corrupts without multiline' test only describes Spark's default behaviour, so it either gets deleted or renamed to say so.

Check: if I break the function under test, does at least one test go red?

## 02 Calling the real function means obeying its parameter contract

read_raw_file builds the path itself: f"{source_path}/{file_name}.csv". The test passed the full file path as source_path, so Spark looked for <tmp>/customers.csv/customers.csv and failed with PATH_NOT_FOUND.

```
source_path = str(tmp_path)     # the FOLDER
file_name   = "customers"       # no .csv -- the function adds it
```

The test was the first caller besides app.py, and it exposed that the parameter name is ambiguous. 'source_path' sounds like a file; 'source_dir' would not.

## 03 Arrange / Act / Assert -- and why the setup lines stay

tmp_path is a new, empty folder for each test. read_raw_file reads from disk, so the test must first create the file it will read.

```
file_name = "customers"                              # Arrange
(tmp_path / f"{file_name}.csv").write_text(...)

df = read_raw_file(spark=spark, source_path=str(tmp_path),   # Act
                   file_name=file_name, schema=customers_schema)

assert df.first()["customer_zip_code_prefix"] == "01310"    # Assert
```

- Keep a blank line between the three parts so they're visible at a glance.
- Set a value once and reuse it (file_name) instead of repeating 'customers.csv' and 'customers' -- two copies that must match are a bug waiting to happen.
- Don't assert the same thing twice: collect()[0] and first() on one row are identical.

**Further reading**

- pytest: anatomy of a test (Arrange-Act-Assert-Cleanup): https://docs.pytest.org/en/stable/explanation/anatomy.html
- pytest: tmp_path fixture: https://docs.pytest.org/en/stable/how-to/tmp_path.html
- Martin Fowler: Given-When-Then: https://martinfowler.com/bliki/GivenWhenThen.html

## 04 Test parameters are fixtures, not arguments

You never call a test function -- pytest does. For every parameter it looks for a fixture with that exact name. Adding file_name: str as a parameter gave:

```
fixture 'file_name' not found
```

```
available fixtures: ..., spark, tmp_path, caplog, ...
```

| Parameter | Where it comes from |
|---|---|
| spark | your fixture in tests/conftest.py |
| tmp_path, caplog | built into pytest |
| file_name | nothing -> error |

Rule: anything in the test's parentheses must be a fixture (or come from parametrize). Plain values like "customers" are local variables inside the test. Fixtures are for things that need setup or cleanup.

**Further reading**
- pytest: how to use fixtures: https://docs.pytest.org/en/stable/how-to/fixtures.html

## 05 parametrize -- one test, many inputs

The decorator tells pytest where the parameter's values come from and runs the test once per value, reporting each run separately.

```
@pytest.mark.parametrize("file_name, schema", SCHEMAS.items())
def test_empty_file_raises_value_error(spark, tmp_path, file_name, schema):
```

- Two names in one string split each (key, value) pair into two parameters.
- Feed it the production source of truth (SCHEMAS) instead of a hand-written list: the hand list missed 3 of 9 tables, and its short names ('customers') weren't real keys, so SCHEMAS[file_name] raised KeyError.
- A new table added to SCHEMAS is tested automatically.
- f"{name}_schema" builds a STRING, not a reference to the variable customers_schema. Look objects up in a dict instead.

When it fits: rules that hold for every table (empty file fails, header mismatch fails). When it doesn't: tests built on table-specific data -- the zip-code test uses a customers header, row and column, so parametrizing only the file name would run the same customers test under different labels.

First draft trap: parametrizing file_name but hard-coding customers_schema made all runs identical. Check that every value that should vary actually varies.

**Further reading**
- pytest: how to parametrize fixtures and test functions: https://docs.pytest.org/en/stable/how-to/parametrize.html

## 06 Testing the error path -- pytest.raises

read_raw_file raises ValueError on an empty file. A test that just calls it lets the error escape -> pytest marks it FAILED, even though raising was correct. A later 'assert df.count() == 0' is never reached; df is never assigned.

```
with pytest.raises(ValueError):
    read_raw_file(spark=spark, source_path=str(tmp_path),
                  file_name=file_name, schema=schema)
```

Inside the block, the expected error = pass; no error = fail. The with block IS the assertion -- no df =, no assert afterwards.

'The job already fails on an empty file -- why test it?'

- The behaviour is the code; the test is the proof. No real file is empty, so this branch had never actually run.
- This exact check broke twice in one session (double log, then no log) while the job still 'failed correctly'. Nothing noticed.

- Refactors (moving read_raw_file to utils) silently drop checks like this.
- It's the one piece of business logic in the module you wrote yourself.

Rule: if your code deliberately raises, test that it raises. Optional extra: the caplog fixture can assert exactly one ERROR record was logged.

**Further reading**

- pytest: assertions about expected exceptions: https://docs.pytest.org/en/stable/how-to/assert.html#assertions-about-expected-exceptions

- pytest: how to manage logging (caplog): https://docs.pytest.org/en/stable/how-to/logging.html

## 07 Build test data from the schema

A parametrized test gets the right schema on every run -- use it instead of a fixed constant. CUSTOMERS_HEADER was only correct for 1 of 9 runs.

```
header = ",".join(schema.fieldNames())
(tmp_path / f"{file_name}.csv").write_text(f"{header}\n")
```

Header-only is the realistic 'empty' file (a source that exported nothing still writes the header), and it's the case where the schema matters.

What the wrong-header experiment revealed (worth keeping):

- enforceSchema=false works: a mismatched header fails. In Python it surfaces as Py4JJavaError wrapping Java's IllegalArgumentException -- use that type in the header-mismatch test.
- A row of only commas (,,,,) counts as 1 row of nulls, so a file of blank rows passes the zero-row check. Open question, separate rule -- not part of this test.

**Further reading**

- PySpark API: StructType (fieldNames): https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.types.StructType.html

- Spark SQL: CSV data source options (multiLine, enforceSchema): https://spark.apache.org/docs/latest/sql-data-sources-csv.html

## 08 Running tests and reading the errors

| Command | Effect |
|---|---|
| python -m pytest | Adds project root to sys.path -> 'import src' works |
| pytest | Doesn't -> ModuleNotFoundError: No module named 'src' |
| pytest -v -k empty_file | Verbose, only tests whose name matches |
| pytest -x | Stop at first failure |

Permanent fix for plain pytest and the VS Code test runner, in pyproject.toml:

```
[tool.pytest.ini_options]
pythonpath = ["."]
testpaths = ["tests"]
```

- Local Spark 3.4 needs Java 17; on Java 21 every test errors at session startup.
- 'collected 0 items / 1 error' = the test FILE failed to import, so nothing ran.
- Type hints aren't checked at runtime, but the name in a hint must exist: a missing 'import StructType' crashed collection with NameError, because annotations are evaluated when the function is defined (Python 3.10-3.13).
- ruff check catches these before pytest does: F821 undefined name, F841 variable assigned but never

used (the leftover df = after removing its assert).

**Further reading**

- pytest: sys.path / import modes: https://docs.pytest.org/en/stable/explanation/pythonpath.html
- pytest: configuration (pyproject.toml): https://docs.pytest.org/en/stable/reference/customize.html
- Ruff rule F821: undefined-name: https://docs.astral.sh/ruff/rules/undefined-name/
- Ruff rule F841: unused-variable: https://docs.astral.sh/ruff/rules/unused-variable/
- Python glossary: function annotation: https://docs.python.org/3/glossary.html#term-function-annotation

## 09 Checklist before asking for a test review

- ruff check tests/ is clean.
- python -m pytest -v -- read the names of what ran, not just the pass count.
- Each test calls my function, not Spark directly.
- The test name and the assertion say the same thing.
- Every parametrized value actually changes the test (schema, not just the label).
- Error cases use pytest.raises; no unused df left behind.
- No commented-out code -- git keeps the old version.